

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 8: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 17 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** Define *optimal substructure* and *overlapping subproblems*. Explain why *both* properties are needed for dynamic programming to be both correct and efficient, and contrast this with divide-and-conquer (e.g. merge sort), which has optimal substructure but *not* overlapping subproblems.
- A2.** Compare *top-down memoization* and *bottom-up tabulation*: how does each avoid recomputing subproblems, what are the practical advantages of each, and why do they have the same asymptotic running time?
- A3.** Explain why the naive recursion for the Fibonacci numbers $F(n) = F(n-1) + F(n-2)$ makes exponentially many calls, while a memoized version makes only $O(n)$. Precisely how many calls does the naive recursion make for $F(n)$ (in terms of Fibonacci numbers)?
- A4.** State the Weighted Interval Scheduling problem precisely. Explain why the earliest-finish-time greedy algorithm (optimal for *unweighted* interval scheduling) is *not* optimal here, and why sorting by weight also fails.
- A5.** Define the function $p(j)$ used in Weighted Interval Scheduling (after sorting by finish time). Explain how to compute all values $p(1), \dots, p(n)$ in $O(n \log n)$ time, and why binary search is applicable.
- A6.** Derive the Weighted Interval Scheduling recurrence $M[j] = \max(M[j-1], w_j + M[p(j)])$ by reasoning about whether interval j is in the optimal solution. Justify why the “take j ” branch may only use intervals $1, \dots, p(j)$.
- A7.** Prove that the recurrence in A6 correctly computes the maximum total weight of a compatible subset of $\{1, \dots, j\}$, by induction on j .
- A8.** Describe how to reconstruct the actual optimal set of intervals (not just its value) from the array M . State the exact condition under which interval j is included during the traceback, and the running time of the reconstruction.

- A9.** State the 0/1 Knapsack problem. Write the 2-D DP recurrence for $dp[i][w]$ (best value using the first i items with capacity w), including the base case and the two cases (item fits vs. does not fit), and explain the take-vs-skip decision.
- A10.** The 0/1 Knapsack DP runs in $O(nW)$ time. Explain carefully why this is *pseudo-polynomial* rather than polynomial. Your answer should refer to the number of bits used to encode W and explain how the running time behaves when that bit-length grows.
- A11.** State the Subset Sum problem and give its Boolean DP recurrence. Explain why, in the space-optimized 1-D version, the inner loop over the target t must run from high to low (rather than low to high), and what would change if you reversed it.
- A12.** Describe how to reconstruct which items were chosen in the 0/1 Knapsack, given the filled 2-D table dp . State the exact traceback condition for “item i was taken.”
- A13.** Contrast the 0/1 knapsack with the *unbounded* knapsack (each item reusable). In the 1-D space-optimized implementations, exactly what differs between the two (loop direction / order), and why does that single difference enforce “at most once” vs. “any number of times”?
- A14.** Give the DP recurrences for both (a) the *minimum number of coins* to make an amount A and (b) the *number of distinct combinations* making A . For the counting version, explain why looping over coins on the *outside* (and amounts inside) counts combinations rather than permutations.
- A15.** State the Rod Cutting problem and its DP recurrence. Explain why it is an instance of *unbounded* knapsack, and describe how to recover the list of cut lengths (not just the maximum revenue).
- A16.** State the Longest Increasing Subsequence (LIS) problem and give the $O(n^2)$ DP recurrence for $L[i]$, the length of the longest increasing subsequence *ending at* index i . Why is the final answer $\max_i L[i]$ rather than $L[n]$?
- A17.** *Discussion (open-ended).* You are given a new optimization problem and suspect it has a DP solution. Describe a systematic process for designing and validating the DP: how you would identify the subproblem and decision, write and prove the recurrence, choose top-down vs. bottom-up, analyze the running time (and check whether it is genuinely polynomial or only pseudo-polynomial), and empirically verify correctness against brute force.

Part B: Quiz

- B1. (True/False)** Dynamic programming requires that a naive recursive solution would re-solve the same subproblems many times (overlapping subproblems); otherwise plain divide-and-conquer is already efficient.
- B2. (Multiple choice)** In Weighted Interval Scheduling (sorted by finish time), $p(j)$ is:
- (a) the latest interval $i < j$ that overlaps j
 - (b) the latest interval $i < j$ that is compatible with j (or 0 if none)
 - (c) the interval with the largest weight among $1, \dots, j - 1$
 - (d) the earliest interval compatible with j
- B3. (Short answer)** Write the Weighted Interval Scheduling recurrence for $M[j]$ in terms of $M[j - 1]$, w_j , and $M[p(j)]$.
- B4. (True/False)** The 0/1 Knapsack DP running time $O(nW)$ is a polynomial-time algorithm in the size of the input.
- B5. (Multiple choice)** In the space-optimized 1-D Subset Sum / 0/1 Knapsack DP, the inner loop over the capacity (or target) is iterated from high to low in order to:
- (a) improve cache performance
 - (b) ensure each item is used at most once
 - (c) allow each item to be used unlimited times
 - (d) avoid integer overflow
- B6. (True/False)** Subset Sum is the special case of 0/1 Knapsack in which each item's value equals its weight and we only ask whether the capacity can be filled exactly.
- B7. (Short answer)** For the unbounded coin-change *counting* DP, should the coin loop be on the outside or inside of the amount loop in order to count distinct combinations (order-independent)?
- B8. (Multiple choice)** Rod Cutting is best classified as:
- (a) 0/1 knapsack (each length usable at most once)
 - (b) unbounded knapsack (each length usable any number of times)
 - (c) weighted interval scheduling
 - (d) a greedy problem with no DP needed
- B9. (Short answer)** In the $O(n^2)$ LIS DP, $L[i]$ is defined as the length of the longest increasing subsequence ending at index i . What is the recurrence for $L[i]$?
- B10. (True/False)** Top-down memoization and bottom-up tabulation, applied to the same recurrence, always compute the same optimal value and have the same asymptotic running time, although memoization may skip subproblems that are never reached.

Part C: Solutions

Part A

- A1. Optimal substructure** means an optimal solution to the whole problem is built from optimal solutions to subproblems: once we fix one decision, what remains is a smaller, independent instance of the same problem whose optimum is used by the global optimum. This is what makes the *recurrence correct*. **Overlapping subproblems** means the naive recursion would solve the same subproblem repeatedly; storing each answer once is what makes the recurrence *efficient*. Both are needed: without optimal substructure the recurrence would be wrong; without overlapping subproblems, memoization saves nothing. Merge sort has optimal substructure (sorted halves combine into a sorted whole) but its subproblems are *disjoint* (different array ranges), so there is nothing to reuse – plain divide-and-conquer already runs in $O(n \log n)$ and DP offers no advantage.
- A2. Top-down memoization** writes the recurrence as a recursive function and caches each subproblem's result on first computation, returning the cache on later calls; subproblems are solved *lazily* (only those reached). **Bottom-up tabulation** fills a table iteratively in an order where each entry depends only on already-computed ones. Memoization is usually easier to write (it mirrors the recurrence) and avoids unreachable subproblems; tabulation has no recursion overhead, makes time/space transparent, and often enables space optimizations (keeping only the last row). They have the same asymptotic running time because both compute each distinct subproblem exactly once, and the total time is (number of distinct subproblems) $\times$ (work per subproblem) in either case.
- A3.** Naive $F(n)$ recursion re-solves overlapping subproblems: computing $F(n)$ calls $F(n-1)$ and $F(n-2)$, and $F(n-1)$ again calls $F(n-2)$, so $F(n-2)$ is computed twice, $F(n-3)$ three times, and so on – the call tree branches like the Fibonacci numbers themselves and has exponentially many leaves. Memoizing computes each $F(k)$ for $k = 0, \dots, n$ exactly once, so $O(n)$ work. The exact number of calls the naive recursion makes for $F(n)$ is $2F(n+1) - 1$ (this is verified empirically in Problem 12).
- A4. Input:** n requests, request i with start s_i , finish f_i ($s_i < f_i$), and weight $w_i \geq 0$; two requests are compatible iff their half-open intervals do not overlap. **Objective:** a compatible set of maximum total weight. The earliest-finish-time greedy is optimal for the *count* (unweighted case) but ignores weights: a single high-value interval can outweigh several low-value compatible ones it conflicts with, so maximizing count is not the same as maximizing weight. Sorting by weight fails too – a heavy interval may block two even-heavier compatible intervals – so no simple greedy ordering is correct, and we use DP.
- A5.** After sorting requests by finish time ($f_1 \leq \dots \leq f_n$), $p(j)$ is the largest index $i < j$ with $f_i \leq s_j$ (the latest interval that finishes before j starts), or 0 if none exists. Since finish times are sorted, the set $\{i : f_i \leq s_j\}$ is a prefix of the index range, so its largest element can be found by **binary search** for s_j in the sorted finish-time array. Each search is $O(\log n)$ and there are n of them, so all $p(j)$ are computed in $O(n \log n)$ (the same as the sort). This is Problem 1.
- A6.** Consider whether interval j (the latest-finishing of $\{1, \dots, j\}$) is in an optimal solution O_j . If $j \in O_j$, then no other member of O_j overlaps j ; since intervals are finish-sorted, every other member has index $\leq p(j)$ (the latest interval compatible with j), so $O_j \setminus \{j\}$ is a

compatible subset of $\{1, \dots, p(j)\}$ – giving value $w_j + M[p(j)]$. If $j \notin O_j$, then O_j is a compatible subset of $\{1, \dots, j-1\}$, giving value $M[j-1]$. The optimum is the larger: $M[j] = \max(M[j-1], w_j + M[p(j)])$. The “take” branch may only use $1, \dots, p(j)$ precisely because any interval with index in $(p(j), j)$ overlaps j and so cannot coexist with it.

A7. By strong induction on j . *Base:* $M[0] = 0$ is correct for the empty instance. *Step* ($j \geq 1$): let O_j be an optimal compatible subset of $\{1, \dots, j\}$ with value $\text{OPT}(j)$. If $j \notin O_j$, then O_j is feasible for $\{1, \dots, j-1\}$, so $\text{OPT}(j) \leq M[j-1]$ by the inductive hypothesis; and any optimum of $\{1, \dots, j-1\}$ is feasible for $\{1, \dots, j\}$, so $\text{OPT}(j) \geq M[j-1]$; thus this branch equals $M[j-1]$. If $j \in O_j$, then (as in A6) $O_j \setminus \{j\}$ is feasible for $\{1, \dots, p(j)\}$ with value $\text{OPT}(j) - w_j \leq M[p(j)]$, so $\text{OPT}(j) \leq w_j + M[p(j)]$; and taking an optimum of $\{1, \dots, p(j)\}$ plus j achieves $w_j + M[p(j)]$, so this branch is attainable. Hence $\text{OPT}(j) = \max(M[j-1], w_j + M[p(j)]) = M[j]$.

A8. Trace back from $j = n$. Interval j is in the optimum iff the “take” branch was at least as good as the “skip” branch, i.e. iff $w_j + M[p(j)] \geq M[j-1]$. When this holds, output j and jump to $j \leftarrow p(j)$ (the latest compatible predecessor); otherwise move to $j \leftarrow j-1$. Repeat until $j = 0$. Each step decreases j , so the reconstruction is $O(n)$. (Ties may be broken either way; both yield optimal sets.) This is Problem 3.

A9. Input: n items with integer weights $w_i \geq 0$ and values $v_i \geq 0$, and integer capacity W ; choose a subset of maximum total value with total weight $\leq W$, each item at most once.
Recurrence: $dp[i][w] =$ best value using items $1, \dots, i$ with capacity w :

$$dp[0][w] = 0; \quad dp[i][w] = dp[i-1][w] \text{ if } w_i > w; \quad dp[i][w] = \max(dp[i-1][w], v_i + dp[i-1][w-w_i]) \text{ otherwise.}$$

If item i does not fit ($w_i > w$) we must skip it. Otherwise we choose the better of skipping it (keep $dp[i-1][w]$) or taking it (gain v_i and recurse on the reduced capacity $w - w_i$ over the first $i-1$ items).

A10. The size of the input – the number of bits to encode it – is about $O(n \log W)$, since W is written in binary using $\approx \log_2 W$ bits (and likewise for each w_i, v_i). The running time $O(nW)$ is polynomial in the *numeric value* W , but $W = 2^{\log_2 W}$ is *exponential* in its bit-length. So if we increase the bit-length of W by one (doubling W), the running time doubles; increasing the bit-length by a factor of two squares W and thus squares the running time. An algorithm polynomial in the numeric values but exponential in the input’s bit-length is **pseudo-polynomial**. It is efficient only when W is small (polynomially bounded in n); 0/1 Knapsack is NP-hard, so no truly polynomial algorithm is known.

A11. Subset Sum: given non-negative integers $a_1, \dots, a_n$ and target T , decide whether some subset sums to exactly T . **Recurrence:** $R[i][t] =$ “some subset of the first i numbers sums to t ,” with $R[0][0] = \text{true}$, $R[0][t > 0] = \text{false}$, and $R[i][t] = R[i-1][t] \vee (t \geq a_i \wedge R[i-1][t-a_i])$. In the 1-D version with a single array **reachable**[], applying a_i by iterating t from T down to a_i ensures that when we read **reachable** $[t-a_i]$ it still reflects the state *before* a_i was applied – so a_i contributes at most once. If we iterated low-to-high, **reachable** $[t-a_i]$ might already include a_i , letting a_i be used multiple times – which would solve the *unbounded* version instead.

A12. Trace back from $dp[n][W]$ with $w = W$. For $i = n$ down to 1: item i was taken iff $dp[i][w] \neq dp[i-1][w]$ (taking it strictly changed the value, meaning the “take” branch won). If taken, record i and set $w \leftarrow w - w_i$; otherwise leave w unchanged. Continue to row $i-1$. This is $O(n)$ and is Problem 7.

- A13.** In *0/1* knapsack each item is used at most once; in *unbounded* knapsack each may be used any number of times. In the 1-D space-optimized implementations the *only* difference is the inner loop direction (or, for counting, the loop nesting): *0/1* iterates the capacity/target from *high to low*, so an item's update reads the table state from *before* this item was applied (each item contributes at most once); unbounded iterates from *low to high*, so an item's update may read entries already updated by the same item in this pass (each item may contribute repeatedly).
- A14.** (a) **Minimum coins:** $dp[0] = 0$ and $dp[a] = 1 + \min_{c_i \leq a} dp[a - c_i]$ (or $+\infty$ if no coin fits), answer $dp[A]$ (or -1 if $+\infty$). (b) **Counting combinations:** $ways[0] = 1$ and, looping over coins on the outside, $ways[a] += ways[a - c]$ for $a = c \dots A$. Putting the coin loop *outside* commits coins in a fixed order, so each combination (multiset) is generated exactly once; if amounts were the outer loop, the same multiset would be counted in every order, counting *permutations* (sequences) instead of combinations.
- A15. Rod Cutting:** given a rod of length n and prices p_ℓ for pieces of length ℓ , cut into integer pieces to maximize total revenue. **Recurrence:** $rev[0] = 0$, $rev[L] = \max_{1 \leq \ell \leq L} (p_\ell + rev[L - \ell])$. It is unbounded knapsack: "item ℓ " has weight ℓ , value p_ℓ , capacity n , and each length may be cut any number of times. To recover the cuts, store for each L the cut length $\ell^*(L)$ achieving the maximum; then starting from $L = n$, repeatedly output $\ell^*(L)$ and set $L \leftarrow L - \ell^*(L)$ until $L = 0$ (Problem 10).
- A16. LIS:** given $a_1, \dots, a_n$, find the length of the longest strictly increasing subsequence (order preserved, elements may be dropped). **Recurrence:** $L[i] = 1 + \max(\{L[j] : j < i, a_j < a_i\} \cup \{0\})$, i.e. extend the best compatible earlier subsequence, or start fresh with length 1. The answer is $\max_i L[i]$, not $L[n]$, because the optimal subsequence may *end* at any index, not necessarily the last one (e.g. if the largest values appear in the middle of the sequence). This is Problem 11.
- A17.** A systematic process: (1) **Identify the subproblem and decision.** Ask "what is the last/first choice the optimal solution makes, and what independent smaller instance remains after it?" Name an array indexed by the relevant parameters. (2) **Write and prove the recurrence.** Express the optimum as a max/min over the possible decisions, then prove correctness by induction (an optimal solution restricted to the subproblem is optimal for the subproblem – optimal substructure). (3) **Choose top-down vs. bottom-up.** Memoize if the recurrence is more natural recursively or many subproblems are unreachable; tabulate if you want transparent time/space or a space optimization. (4) **Analyze the running time** as (number of subproblems) $\times$ (work per subproblem), and check whether it is genuinely polynomial in the *input size* or only pseudo-polynomial (polynomial in a numeric value like W or A). (5) **Verify empirically.** Implement a brute-force solver (exponential, for small inputs) and compare against the DP on many random instances and edge cases – exactly as Problems 5, 6, 8, 9, 10, and 11 do this week. A single mismatch should be minimized by hand to expose the flaw in the recurrence.

Part B

- B1. True.** Without overlapping subproblems there is nothing to reuse, and plain divide-and-conquer is already efficient (e.g. merge sort). DP's advantage comes precisely from caching answers to subproblems that recur.

- B2. (b)** the latest interval $i < j$ that is compatible with j (finishes at or before j starts), or 0 if none. This is found by binary search on the sorted finish times.
- B3.** $M[j] = \max(M[j-1], w_j + M[p(j)])$, with $M[0] = 0$.
- B4. False.** $O(nW)$ is *pseudo*-polynomial: it is polynomial in the numeric value W , but W is exponential in its bit-length ($\approx \log_2 W$ bits), so the algorithm is exponential in the input size. 0/1 Knapsack is NP-hard.
- B5. (b)** to ensure each item is used at most once. Iterating high-to-low means $dp[w - w_i]$ still holds the pre-item value, so the item contributes once; low-to-high would allow reuse (the unbounded version).
- B6. True.** Subset Sum is exactly 0/1 Knapsack with $v_i = w_i$ and $W = T$, asking only whether the optimum equals (can reach) T .
- B7. Outside.** The coin loop must be on the outside (amounts inside) to count order-independent combinations; swapping the order would count ordered sequences (permutations).
- B8. (b)** unbounded knapsack – each piece length may be used any number of times, with capacity equal to the rod length.
- B9.** $L[i] = 1 + \max(\{L[j] : j < i \text{ and } a_j < a_i\} \cup \{0\})$; the answer is $\max_i L[i]$.
- B10. True.** Both evaluate the same recurrence, computing each distinct subproblem once, so they return the same value with the same asymptotic running time; memoization additionally skips subproblems that are never reached from the top-level call.